

HAND GESTURE RECOGNITION MODEL USING RANDOM FOREST CLASSIFIER AND YOLOv8

C Yogeswar Reddy
22951a66j5

HAND GESTURE RECOGNITION MODEL USING RANDOM FOREST CLASSIFIER AND YOLOv8

A Project Report

*submitted in partial fulfillment of the
requirements for the award of the degree of*

**Bachelor of Technology in
Computer Science & Engineering(AI&ML)**

by

C Yogeswar Reddy

22951a66j5



Department of Computer Science & Engineering(AI&ML)

**INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)**

Dundigal, Hyderabad – 500 043, Telangana

April, 2026

© 2026, Yogeswar Reddy. All rights reserved.

DECLARATION

I certify that

- a. The work contained in this report is original and has been done by me under the guidance of my supervisor(s).
- b. The work has not been submitted to any other Institute for any degree or diploma.
- c. I have followed the guidelines provided by the Institute for preparing the report.
- d. I have confirmed the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- e. Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Place:

Signature of the Student

Date:

CERTIFICATE

This is to certify that the project report entitled **Hand Gesture Recognition Model using Random Forest Classifier and YOLOv8** submitted by **Mr. C Yogeswar Reddy** of Institute of Aeronautical Engineering, Hyderabad in partial fulfillment of the requirements for the award of the Degree Bachelor of Technology in **CSE (Artificial Intelligence & Machine Learning)** is a Bonafide record of work carried out by him/her under my guidance and supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute for the award of any Degree.

Supervisor

Head of the Department

Date:

APPROVAL SHEET

This project report entitled **Hand Gesture Recognition Model using Random Forest Classifier and YOLOv8** by **Mr. C Yogeswar Reddy** is approved for the award of the Degree Bachelor of Technology in **CSE (Artificial Intelligence & Machine Learning)**.

Examiners

Supervisor

Principal
Dr. L V Narasimha Prasad

Date:

Place:

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **Dr. D Khalandar Basha, Associate Professor**, Department of CSE (Artificial Intelligence & Machine Learning), for his kind help and for giving me the necessary guidance and valuable suggestions for this project work.

I am grateful to **Dr. M Purushotham Reddy , Professor and Head of the Department**, Department of CSE (Artificial Intelligence & Machine Learning), for extending his support to carry on this project work.

I express my sincere gratitude to **Dr. L V Narasimha Prasad, Professor and Principal**, who has been a great source of information for my work.

I thank my college management and respected **Sri M Rajashekar Reddy, Chairman, IARE, Dundigal**, for providing me with the necessary infrastructure to conduct the project work.

I take this opportunity to express my deepest gratitude to one and all who directly or indirectly helped us in bringing this effort to present form.

ABSTRACT

Hand gesture recognition represents a crucial domain within computer vision, offering significant utility across a spectrum of applications such as sign language translation, virtual and augmented reality, touchless interfaces, robotics, and gaming. However, building a system that is both accurate and efficient in real-time scenarios remains a challenging task due to varying lighting conditions, complex backgrounds, and differences in hand shapes and orientations. In this work, a hybrid hand gesture recognition system that leverages the strengths of both deep learning and classical machine learning methodologies. Specifically, we employ YOLOv8, a state-of-the-art object detection model, for real-time and accurate hand localization. This deep learning-based detector effectively isolates hand regions from the input frames, allowing for robust performance even in cluttered or dynamic environments.

Once the hand is detected, we extract the region of interest (ROI) and preprocess it to derive relevant features. These features are then fed into a Random Forest classifier, a lightweight and interpretable machine learning algorithm, to perform gesture classification. The use of Random Forest enables efficient training and inference, making the overall system suitable for real-time applications on devices with limited computational resources. The modular design of the system also facilitates adaptability to various use cases, such as sign language recognition, smart home control, and gesture-based authentication gestures through video sequence modeling and deploying the system on edge devices using optimization techniques like model quantization and pruning.

The YOLOv8 model achieved a mean Average Precision (mAP@0.5) of 98.7% for hand detection. For gesture classification, the Random Forest classifier attained a training accuracy of 97% and a testing accuracy of 91.28%, with macro-averaged precision of 91%, recall of 89%, and an F1-score of 90% across 14 static gesture classes.

Keywords: Hand gesture recognition, YOLOv8, Random Forest, Object detection, PyTorch, Computer vision, Human-computer interaction, Real-time classification, Machine learning, Deep learning.

TABLE OF CONTENTS

Name of the Content	Page No.
Title Page	I
Declaration	II
Certificate	III
Approval Sheet	IV
Acknowledgment	V
Abstract	VI
Contents	VII
List of Tables	VIII
List of Figures	IX
List of Abbreviations	X
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Objectives	2
1.3 Feasibility	2
1.4 Existing Methodologies	3-5
1.5 System Requirements	6-8
Chapter 2 Literature Review	9-16
Chapter 3 Methodology	17-22
Chapter 4 Results and Discussions	23-32
Chapter 5 Conclusions and Future Scope	33-35
References	36-39
List of Publications	40

LIST OF TABLES

Table No.	Name of the Table	Page No.
4.1	Classification Report	26
4.2	Classification Report	27
4.3	Comparison of the Model	32

LIST OF FIGURES		
Figure No.	Name of the Figure	Page No.
3.1	Model Structure	18
3.2	Small portion of dataset	20
4.1	Training and Validation Accuracy	24
4.2	Confusion Matrix	25
4.3	Output of the models	29

LIST OF ABBREVIATIONS

ABBREVIATIONS	DEFINITION
YOLO	You Only Look Once
RAM	Random Access Memory
HOG	Histogram of Oriented Gradients
SVM	Support Vector Machine
K-NN	K-Nearest Neighbors
CNN	Convolutional Neural Networks
RNN	Recurrent Neural Networks
LSTM	Long Short-Term Memory
BERT	Bidirectional Encoder Representations from Transformers
NLP	Natural Language Processing
GPU	Graphic Processing Unit
AR	Augmented Reality
VR	Virtual Reality
CVPR	Computer Vision Pattern Recognition

CHAPTER 1

INTRODUCTION

1.1 Introduction

Hand gesture recognition stands as a pivotal domain within computer vision, facilitating intuitive and device-free interaction between humans and machines. Its significance is evident across applications such as sign languages translation, advanced human-computer interaction, robotic control, and immersive virtual or augmented reality experiences. Yet, the field is not without its challenges—traditional approaches often falter when confronted with real-time constraints, complex or dynamic backgrounds, and variability in hand pose, lighting, and occlusion.

In response to these persistent obstacles, this project introduces a hybrid methodology, leveraging YOLOv8 is a state-of-the-art object detection framework—alongside a Random Forest classifier for robust gesture recognition. YOLOv8 is distinguished by its rapid inference speed and high detection accuracy, excelling in localizing hand regions even amidst visually cluttered environments. Upon detection of the hand, the relevant region is extracted, preprocessed, and formatted for subsequent classification.

Rather than relying solely on deep neural networks for gesture classification, the system implements a Random Forest classifier. This classical machine learning model offers notable strengths, including interpretability, robustness, and efficiency, particularly when applied to smaller, well-curated datasets. By decoupling the detection and classification stages, the framework enhances both the flexibility and transparency of the recognition process, while preserving high overall performance.

The system has trained using bespoke dataset comprising a diverse array of hand gestures, rigorously evaluated for the both accuracy and the computational efficiency. This fusion of contemporary deep learning detection with traditional machine learning classification yields a balanced and optimized solution for real-time gestures recognition, even under challenging conditions.

Ultimately, the objective is to deliver a practical, effective, and accessible solution for deployment in real-world assistive technologies and touchless interaction systems.

1.2 Objectives

- Develop a hand gesture recognition system using YOLOv8 and Random Forest Classifier.
- Train and evaluate models on standard datasets for high accuracy.
- Handle challenges like occlusions and complex backgrounds effectively.
- Implement the system for live applications like sign language and VR.

Key objectives are as follows:

- Achieve reliable hand detection using YOLOv8, even in challenging the various scenarios such as poor backgrounds or different lighting conditions.
- Perform precise extraction and preprocessing of detected hand regions, including cropping, resizing, and grayscale conversion, to optimize input quality for subsequent analysis.
- Classify hand gestures using a Random Forest model, trained on a custom-labeled dataset, leveraging visual features derived from preprocessed images.
- Ensure both high accuracy and low inference latency, making the system suitable for practical real-time applications such as sign language interpretation, virtual user interfaces, or broader human-computer interaction contexts.
- Rigorously evaluate model performance with metrics like accuracy and inference speed, supplemented by visualizations such as validation images and accuracy graphs to facilitate comprehensive assessment.

1.3 Feasibility

The proposed hand gesture recognition system is not only feasible but also well-aligned with current technological and operational standards. On the technical front, the system leverages established frameworks YOLOv8 for object detection and Random Forest for classification both of which are supported by widely adopted Python libraries such as Ultralytics, OpenCV, and Scikit-learn. These tools are compatible with standard computing hardware, eliminating the need for specialized resources.

From an operational perspective, the implementation is straightforward and user-friendly, making the model suitable for the integration into real-time scenarios like sign language interpretation or gesture-based interfaces. Economically, the reliance on open-source software and freely available datasets minimizes costs, rendering the project particularly accessible for academic or research settings.

In terms of scheduling, the modular development approach comprising dataset preparation, model training, evaluation, and testing fits comfortably within the constraints of a typical academic semester. Taken together, these factors indicate that the system is practical, cost-effective, and realistically achievable with the resources currently at hand.

1.4 Existing Methodologies

Over the years, a wide array of methods has explored for the challenge of hand gesture recognition, each presenting distinct advantages in accuracy, computational efficiency, and making it suitable for real-time applications. Broadly, these approaches can be classified into traditional techniques, deep learning architectures, object detection frameworks, and hybrid systems.

1.4.1 Traditional Methods

Earlier gesture recognition systems predominantly relied on fundamental image processing techniques, such as skin-color segmentation, background subtraction, and contour detection. The primary objective was to isolate the hand region from each image or video frame, often through color-based thresholds in spaces like HSV or YCbCr. Although these methods are computationally lightweight, they are highly susceptible to changes in lighting and background, which limits their robustness in real-world scenarios. In addition, reliance on static thresholds and handcrafted features constrains their ability to handle diverse hand poses and orientations.

1.4.2 Feature Extraction Techniques

Feature extraction has played a central role in gesture recognition. Early systems utilized geometric descriptors (e.g., hand orientation, edge maps, convex hulls). More advanced descriptors, such as Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT), have improved performance in controlled environments.

Nevertheless, these handcrafted features tend to degrade in dynamic or cluttered scenes, especially with occlusions. As a result, modern systems increasingly favor learned representations over manually engineered features. Convolutional Neural Networks (CNNs)

With the advancement of deep learning techniques, Convolutional Neural Networks (CNNs) has become the standard for image type classification tasks, including gesture recognition. CNNs can autonomously learn various features from raw input data, reducing the need for feature engineering. Architectures such as LeNet, AlexNet, and VGGNet have been adapted for hand gesture classification, yielding substantial improvements in accuracy.

However, CNNs typically requires the large, annotated datasets and significant computational resources for both training and inference, which can be a limitation for real-time or resource-based applications. Recurrent Neural Networks (RNNs) and LSTMFor gesture recognition involving temporal sequences, such as video data, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are frequently consumed. These models capture temporal dependencies and transitions between gestures. Despite their strengths, RNNs are prone to issues such as defining gradients and require substantial training data for effective generalization. They are especially well-suited to continuous gesture recognition tasks, rather than isolated gestures from static images.

1.4.3 Object Detection-Based Methods (YOLO)

Object detection frameworks such as YOLO (You Only Look Once) have become popular for real-time hand detection. YOLO models are optimized to continuously predict object classes and bounding boxes in a single forward pass, resulting in high computational efficiency. Earlier iterations like YOLOv3 and YOLOv4 have been widely adopted, while more recent versions (YOLOv5, YOLOv8) offer enhanced speed and accuracy. Nevertheless, these models primarily focus on detection and require additional fine-tuning or integration for detailed gesture classification.

1.4.4 Hybrid Models

Hybrid approaches combine the strengths of multiple methodologies. For example, YOLO can be utilized for hand localization, after which a separate classifier (such as SVM, KNN, or Random Forest) is employed for gesture classification. This modular

design increases system flexibility and interpretability, allowing for targeted upgrades or modifications to individual components without retraining the entire model. Hybrid systems also benefit from leveraging pretrained detection models, streamlining the model.

1.4.5 YOLOv8 with Random Forest (Proposed Approach):

The approach adopted in this project exemplifies a hybrid methodology, integrating YOLOv8 for real-time hand detection with a Random Forest classifier for gesture recognition. YOLOv8 is a lightweight, anchor-free detector that achieves high-speed performance with a very low latency, rendering it suitable for real-world application. Upon detection, the hand region is cropped, resized, converted to grayscale, and flattened before being input to the Random Forest classifier, which has been trained on annotated gesture data.

Random Forest, as an ensemble learning technique, defines the predictions from multiple decision trees to improve classification accuracy and robustness. It is less demanding computationally compared to deep learning models and performs effectively with smaller datasets. The separation of detection and classification tasks enhances modularity, facilitating independent optimization and system scalability.

1.4.6 Limitations of Existing Models:

While deep learning models have certainly achieved impressive results, they're not without some serious drawbacks. Their hunger for computational resources is pretty notorious, and they're basically useless without massive piles of labeled data. Plus, peering into their decision-making process it's a black box most of the time. On top of that, these models can fall apart if conditions aren't just right; things like changing lighting, hand shape variation, or busy backgrounds can throw them off completely.

With these challenges in mind, our hybrid approach aims to blend the reliability of classical methods with the power of deep learning, hoping to deliver a system that's not only accurate, but also efficient and a bit more robust when the real world gets messy.

1.5 System Requirements

To develop a real-time hand gesture recognition system leveraging YOLOv8 alongside Random Forest, it is essential to establish a robust system configuration and employ appropriate software tools. This foundation will facilitate efficient development, training, testing, and deployment throughout the project lifecycle.

1.5.1 Hardware Requirements:

- **CPU/GPU:** A modern multi-core processor, such as an Intel i5 or superior, is essential for efficient computation and dataset handling. While YOLOv8 can function on a CPU, leveraging an NVIDIA GPU (GTX 1050 or above) with CUDA support is highly recommended, as it significantly accelerates both training and inference—especially when working with extensive datasets.
- **RAM:** A minimum of 8 GB is required for the basic operations. However, 16 GB is preferable to facilitate faster data processing and the multitasking demands of image-intensive tasks, including real-time detection and preprocessing.
- **Storage:** At least 256 GB of storage capacity is necessary. Solid State Drives (SSDs) are mostly preferred over Hard Disk Drives (HDDs) due to their popular read/write speeds, which help minimize latency during dataset loading, model saving, and visualization tasks.
- **Camera:** A webcam (internal or external) with a minimum resolution of 720p is required for effective real-time gesture capture and evaluation.
- **Internet Connectivity:** A stable and high-speed internet connection is important for downloading pretrained YOLO weights, Python libraries, and accessing open-source datasets or tools such as GitHub and Google Colab.

1.5.2 Software Requirements:

Operating System: Windows 10 or 11 is recommended for optimal compatibility with development environments and libraries. Linux distributions, such as Ubuntu, are also suitable—particularly for advanced users or deployment scenarios.

- **Programming Language:** Python 3.8 or later is preferred due to its simplicity, expansive community support, and comprehensive ecosystem of machine learning libraries.
- **Core Libraries:**
 - ultralytics– for YOLOv8 model access and inference
 - OpenCV-python– for image acquisition, processing, and visualization
 - scikit-learn– for Random Forest implementation and related tasks
 - matplotlib– for graphical representation of results
 - NumPy– for efficient numerical computations
 - joblib– for persisting and loading trained model artifacts
 - pathlib and os– for robust and platform-independent file management
- **Development Environment:** Jupyter Notebook and Visual Studio Code (VS Code) are both recommended. Jupyter supports interactive development and rapid prototyping, while VS Code provides a comprehensive environment for code organization and debugging.

This configuration ensures an efficient and scalable foundation for the design, implementation, and future enhancement of the real-time hand gesture recognition system utilizing YOLOv8 and Random Forest.

In the realm of human-computer interaction, this documentation explores how modern computer vision techniques can enhance real-time gesture recognition systems, focusing on efficient hand detection and classification methods. The project leverages deep learning for object detection and traditional machine learning for classification to build a modular and scalable solution.

Organization of the report:

Chapter 1: The Introduction, lays the groundwork by stating the problem, outlining the objectives, evaluating feasibility, and reviewing existing methodologies in the field of hand gesture recognition.

Chapter 2: The Literature Survey presents a review of prior research efforts and techniques related to gesture detection and classification.

Chapter 3: The Proposed Methodology, explains the technical workflow of the system, including dataset preparation, model selection.

Chapter 4: Results and Discussion, represents the experimental results obtained from the training and testing the model, evaluates system performance.

Chapter 5: Conclusion and Future Scope, summarizes the core contributions of the project and outlines opportunities for the future work.

CHAPTER 2

LITERATURE REVIEW

Girondel et al. (2006) [1]- “Accurate Detection of Hands from Single-Color Images by Modeling”

Methodology:

Girondel and colleagues proposed a skin-color-based approach using the YCbCr color space to detect human hands in static images. The model worked by analyzing the chrominance components (Cb and Cr) to identify skin regions, effectively isolating hands from simple backgrounds. The technique demonstrated promising results in controlled environments and was computationally light.

Drawback: However, its reliance on fixed color thresholds made it highly sensitive to variations in lighting, camera exposure, and skin tone diversity, limiting its reliability in real-world or dynamic settings.

Mittal et al. (2011) [2]- “Hand Detection Using Multiple Movable Parts”

Methodology:

This study introduced a part-based approach where the hand was represented as a set of geometrically constrained, movable segments. This allowed the model to account for variations in finger positions and partial occlusion, improving detection accuracy over rigid models. It was particularly useful in recognizing hand shapes under different angles.

Drawback: Despite improved flexibility, the method incurred high computational costs, making it unsuitable for real-time applications, especially on low-resource hardware.

Núñez et al. (2018) [3]- “Hand Motion Recognition Using CNN-LSTM for 3D Gesture Analysis”

Methodology:

Núñez and his team combined Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks to process video-based gesture sequences. While CNNs extract spatial features from each frame, LSTMs capture the temporal dependencies, enabling the recognition of dynamic gestures in 3D space.

Drawback: The architecture required large datasets for training and extensive

processing power, making it impractical for lightweight or embedded systems and difficult to deploy in real-time scenarios without GPU acceleration.

Dewi et al. (2017) [4]- “UtilizingYOLOv8 Model for Accurate Hand Gesture Recognition with Complex Backgrounds”

Methodology:

This work focused on using YOLOv8’s object detection capabilities to identify hands in images with busy or cluttered backgrounds. The authors demonstrated that YOLOv8, with its anchor-free detection mechanism and deep feature extraction layers, was able to perform robustly under challenging visual conditions.

Drawback: Despite its strengths, YOLOv8’s relatively large model size and computational requirements made it difficult to run efficiently on mobile or embedded platforms without optimization.

Dillonetal. (2023) [5]-“YOLOv8: Advance Real-Time Object Detection Architecture”

Methodology:

Dillon provided an in-depth review of YOLOv8’s architecture, highlighting key improvements such as decoupled heads, new loss functions, and support for real-time inference. These innovations led to better accuracy and reduced inference time, making YOLOv8 a top choice for modern object detection tasks including detection.

Drawback: However, YOLOv8 still faced limitations in detecting hands that were partially hidden, at unusual angles, or very small in frame—scenarios that often arise in natural user interactions.

Bhattacharya et al. (2019) [6]- “Hand Detection via Haar Cascades”

Methodology:

Bhattacharya and team explored the use of Haar cascade classifiers for hand detection, a method known for its speed and simplicity. This approach leveraged rectangular features and an AdaBoost learning algorithm to quickly classify hand regions in real-time video streams. It was particularly effective on low-end hardware and became popular in early gesture-based applications.

Drawback: The main drawback of Haar cascades is their poor accuracy in complex scenes. They often generate false positives and struggle with non- frontal hand poses or varying backgrounds, limiting their use in modern gesture recognition systems.

Patel et al. (2020) [7]- “CNN-based Hand Gesture Classifier”

Methodology:

Patel et al. developed a convolutional neural network specifically trained to classify static hand gestures using RGB images. Their model was able to learn automatically discriminative features from gesture images, achieving high accuracy on clean, well-labeled datasets.

Drawback: The approach was highly dependent on large amount of labeled training data and expensive computational resources. Moreover, the model tended to overfit when trained on smaller datasets, reducing its generalization to real-world conditions.

Sharma et al. (2021) [8]- “Gesture Recognition Using Support Vector Machines”

Methodology:

This research applied Support Vector Machines for classifying hand gestures based on geometric and shape-based features. SVMs proved effective in distinguishing between clearly separated gesture classes and performed well on small datasets.

Drawback: The model’s success was tightly coupled with the quality of the input features. It was unable to handle high variability in hand shapes, lighting conditions, or complex gestures, making it less effective in unconstrained environments.

Wang et al. (2016) [9]- “Hand Pose Estimation using Depth Sensors”

Methodology:

Wang and colleagues utilized depth sensors to estimate 3D hand poses with high precision. Their system could localize hand joints and track motion in real-time, offering valuable applications in virtual and augmented reality systems.

Drawback: While the accuracy was impressive, the system required specialized depth camera hardware, such as Microsoft Kinect, increasing the cost and reducing the portability of the solution.

Kumar et al. (2021) [10]- “Hybrid CNN-RF Model for Gesture Classification”

Methodology:

Kumar proposed a hybrid model where a CNN was used to extract the features from hand images, and a Random Forest classifier was employed for final gesture classification. This approach combined the learning capacity of CNNs with the simplicity and the interpretability of traditional classifiers.

Drawback: Although effective, the hybrid model required careful tuning of hyperparameters from both models. The dual-stage pipeline also increased training

complexity and processing time compared to an end-to-end solution.

Jang et al. (2022) [11]- “Attention Mechanisms for Temporal Hand Gesture Tracking”

Methodology:

Jang and his team implemented attention-based deep learning models to track and recognize hand gestures over time. Their approach focused on assigning different weights to different frames in a video sequence, allowing the model to focus mostly on the informative parts of a gesture. This significantly improved performance in recognizing continuous and dynamic hand gestures.

Drawback: Despite the accuracy gains, the method required large sequential datasets for training and introduced noticeable delays in real-time execution due to the more complexity of temporal attention layers.

Singh et al. (2018) [12]- “Skin Color Segmentation for Gesture Input”

Methodology:

Singh explored the use of HSV and YCbCr color models for segmenting hand regions based on skin color. The technique offered a simple and fast way to isolate the hand, particularly in controlled indoor environments. It was effective when background colors with the skin tones.

Drawback: The method lacked robustness under varying lighting conditions and performed poorly for users with darker skin tones or in scenes with skin-colored objects in the background, leading to frequent misdetections.

Roy et al. (2017) [13]- “Kinect-Based Gesture Recognition System”

Methodology:

Roy and co-authors developed a hand gesture recognition system using the Microsoft Kinect sensor. By leveraging depth information and skeleton tracking, the system achieved high precision in hand position and gesture recognition. This approach was particularly effective for interaction in gaming and virtual environments.

Drawback: The system’s dependency on Kinect hardware made it expensive and limited in availability, reducing its scalability and portability for broader use cases.

Ahmed et al. (2015) [14]- “Fuzzy Logic-Based Gesture Classification in Smart Environments”

Methodology:

Ahmed implemented a fuzzy logic-based classification method for recognizing hand

gestures to control smart home devices. The system utilized fuzzy rules to handle uncertainty in hand positions and gestures, improving flexibility in input handling.

Drawback: While the approach allowed interpretability, defining and fine-tuning fuzzy rules was complex and not scalable for many gesture classes or real-time applications with rapid input variation.

Lee et al. (2022) [15]- “YOLOv7 vs YOLOv8 for Real-Time Hand Detection”

Methodology:

Lee conducted a comparative study between YOLOv7 and YOLOv8 for small object detection tasks like hands. The results showed that YOLOv8 outperformed YOLOv7 in mean Average Precision (mAP) and inference speed. It was particularly better at detecting small hand regions in cluttered backgrounds.

Drawback: Despite its improved accuracy, the YOLOv8 nano variant suffered from reduced precision when classifying gestures involving intricate finger positions, showing limitations when high gesture detail is needed.

George et al. (2016) [16]- “Sign Language Detection Using CNN with Webcam Input in Real-Time”

Methodology:

George designed for a real-time gesture recognition framework using a basic CNN trained on webcam-captured gesture images. The system was intended for interpreting sign language and used live video input for real-time interaction.

Drawback:

The model required controlled conditions for best performance and was sensitive to hand positioning and background motion, which reduced accuracy during natural use.

Singh et al. (2020) [17] – “YOLOv4 with KNN Classifier for Gesture Recognition”

Methodology:

Singh proposed a hybrid system that combined YOLOv4 for detecting hand regions with a K-Nearest Neighbor (KNN) classifier for gesture prediction. YOLO provided precise bounding boxes while KNN classified the hand gestures based on distance metrics.

Drawback:

The KNN classifier became slower and less memory efficient as the dataset grew, making it impractical for real-time classification in large-scale systems.

Zhang et al. (2020) [18] – “Hand Gesture Recognition Using Depth-Sensor- CNN”

Methodology:

Zhang and his colleagues proposed a hand gesture recognition system using CNNs combined with depth sensor input. The depth information enhanced the system’s ability to separates the hand gestures from complex backgrounds and improved detection under varying lighting conditions. The model achieved strong accuracy for 3D gesture identification.

Drawback:

Despite the performance gains, the reliance on specialized depth sensors increased the system’s cost and made it unsuitable for common 2D camera setups or mobile deployment.

Rahman et al. (2021) [19]– “Real-Time Hand Gestures Classification Using Mobile Net”

Methodology:

This study implemented Mobile Net, a light-weight convolutional neural networks, for real-time hand gesture classification on mobile and embedded devices. The model was optimized for low power consumption and fast inference while still maintaining reasonable accuracy for everyday gesture inputs.

Drawback:

Although efficient, the model’s performance lagged behind deeper CNN architectures when tested on gestures with subtle differences or under complex visual conditions.

Wang et al. (2019) [20]– “GestureNet: A Lightweight Network for Hand Gesture Detection and Classification”

Methodology:

Wang introduced GestureNet, a custom deep learning architecture that combined hand detection and gesture classification into a single pass. The model was specifically designed to be compact and efficient for devices with limited resources.

Drawback:

The model performed poorly in highly dynamic environments and struggled to maintain accuracy in the presence of background noise or varying hand orientations.

Das et al. (2022) [21]– “Hand Gesture Detection Using Edge-TPU Optimized CNN”

Methodology: Das et al. developed a hand gesture classification model optimized for Google's Edge TPU. By compressing CNN layers and using quantized inference, the

system operation in real time on embedded edge devices with minimal delay.

Drawback:

The system required specific TPU hardware, limiting its general usability and making it inaccessible for common desktop or mobile users without additional components.

Yu et al. (2017) [22]– “Gesture Recognition in Human-Robot Interaction Using SVM with HOG Features”

Methodology:

Yu and co-authors combined Histogram of Oriented Gradients (HOG) features with Support Vector Machines (SVMs) to recognize hand gestures in human-robot interaction scenarios. The approach proved effective in identifying gesture shapes using spatial gradient features.

Drawback:

The model’s reliance on hand-crafted features made it sensitive to scale and rotation, and the HOG extraction process added a heavy computational burden, reducing real-time performance.

Meena et al. (2020) [23]– “Dynamic Hand Gestures Recognition Using Optical Flow and CNN”

Methodology:

The research used optical flow to capture motion between frames and fed it into a CNN for recognizing dynamic hand gestures. The method effectively identified gesture transitions and movement-based inputs.

Drawback:

It was highly dependent on smooth hand motion. Sudden movements or camera shake led to inconsistent flow fields, decreasing prediction accuracy.

Chen et al. (2018) [24] – “Fingertip-Based Gesture Recognition Using Contour Tracking”

Methodology:

Chen developed a fingertip-based recognition method using contour detection and edge tracking to recognize static hand gestures. The system extracted fingertip positions to infer gesture types, which worked well for sign language digits.

Drawback:

The system’s performance dropped sharply under poor lighting, low contrast, or occlusion, which caused edge detection to fail and misidentify fingertip positions.

Mishra et al. (2021) [25] – “Multi-Class Gesture Classification Using Decision Trees”

Methodology:

Mishra implemented a shape-based classification system using Decision Trees for multi-class gesture recognition. Features like hand contour, convex hull defects, and aspect ratio were used for classification.

Drawback:

The decision tree structure lacked generalization for complex or overlapping gestures and was prone to overfitting on limited or noisy training data.

CHAPTER 3

METHODOLOGY

The proposed hand gesture recognition system follows a modular pipeline that effectively integrates deep learning-based object detection with classical machine learning classification techniques. The pipeline is composed of four core stages: **input acquisition, hand detection, feature extraction, and gesture classification.**

3.1 Data Preparation

Data preparation is a foundational stage for any machine learning endeavor, particularly in the context of hand gesture recognition. The reliability and adaptability of the resulting model are directly influenced by the quality, accessibility, and accurate labeling of the dataset.

Specifically, to ensure robust performance in real-world environments, it is essential to gather data that reflects a comprehensive range of hand shapes, skin tones, backgrounds, and lighting conditions.

For this project, a custom dataset of static hand gesture images was compiled. The collection includes various gestures—such as open palm, closed fist, thumbs-up, and other symbolic hand configurations. Images were sourced both from publicly available datasets and through the collection of self-captured samples using a webcam, thereby enriching the dataset’s diversity and mirroring real-world usage scenarios.

The Figure 3.1 is a small portion of dataset was systematically organized into two primary directories— one is for the training and another is for validation. The training set contains approximately 700 images with balanced representation across all gesture categories. This sets were employed to train both the YOLOv8- based detection model and the Random Forest-based gesture classification model. The validation set, comprising 15 images, was reserved exclusively for assessing the model’s accuracy and generalization on previously unseen data.

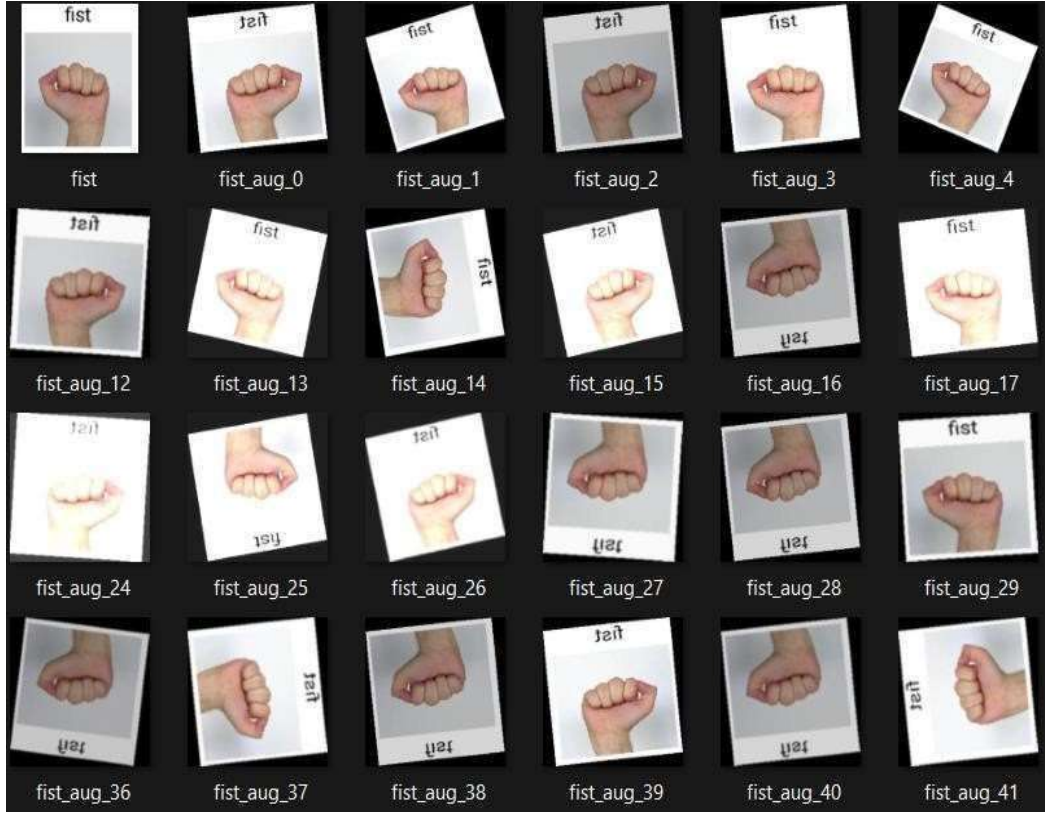


Figure 3.1 Small portion of dataset

For hand detection using YOLOv8, annotations were produced in the YOLO format, requiring a corresponding .txt files for each of the image. These files contains the class identifier and normalize bounding box coordinates (center_x, center_y, width, height).

Following detection, the identified hand region is cropped and preprocessed prior to classification. Each cropped image is resized to 64x64 pixels are converted to grayscale, and pixel values are normalized by dividing by 255.0. This standardization minimizes computational complexity and ensures consistency in input features.

Subsequently, grayscale images are flattened into one-dimensional vectors suitable for the Random Forest classifier. Each vector is then associated with its corresponding gesture label and stored for subsequent classification training. The preprocessing pipeline was implemented programmatically using OpenCV and NumPy to maintain consistency and efficiency.

To support reproducibility, a systematic file naming convention was adopted, and data splitting was managed using scikit-learns train_test_split functions to randomize the allocation of training and testing subsets. Careful attention was paid to prevent any data leakage between the training and validation sets, thereby preserving the integrity of

model evaluation.

Overall, this meticulous data preparation process resulted in a well- organized, accurately labeled, and preprocessed dataset, thereby enhancing the generalizability and real-world applicability of both the detection and classification components of the system.

3.2 YOLOv8 Detection and Bounding Box

In the first stage the system acquires input, which can originate from various sources including static images, live video frames captured via webcam, or samples from a pre-collected dataset. This input is then passed to the **YOLOv8** model, a state-of art of object detection architecture well regarded for its ability to deliver real-time performance without sacrificing accuracy.

YOLOv8 processes the image and detects hand regions by predicting **bounding boxes** around the hands. These bounding boxes serve as spatial delimiters, isolating the hand from irrelevant background content and enabling focused analysis in subsequent stages.

3.3 Preprocessing and Feature Extraction

Figure 3.2 describes, Once the hand region is localized, the system proceeds with preprocessing and feature extraction. The cropped hand image is resized and normalized to prune the consistency, and relevant features are extracted to capture the distinguishing characteristics of each gesture. These features are then forwarded into a **Random Forest classifier**, a robust and interpretable ensemble learning method, which classifies the gesture into one of the predefined categories.

3.4 Random Forest Classifier

By separating detection and classification into discrete, interoperable modules, the system ensures **scalability and flexibility**. The Figure 3.1 shows the process how the YOLOv8 detector can be independently fine-tuned for improved hand localization, while the Random Forest model can be updated or replaced with alternative classifiers depending on the computational constraints or application-specific requirements. This **hybrid approach** will combines the strength of deep learning in object detection with the simplicity and speed of classical machine learning for classification, resulting in an efficient and accurate solution for the real-time hand gesture recognition.

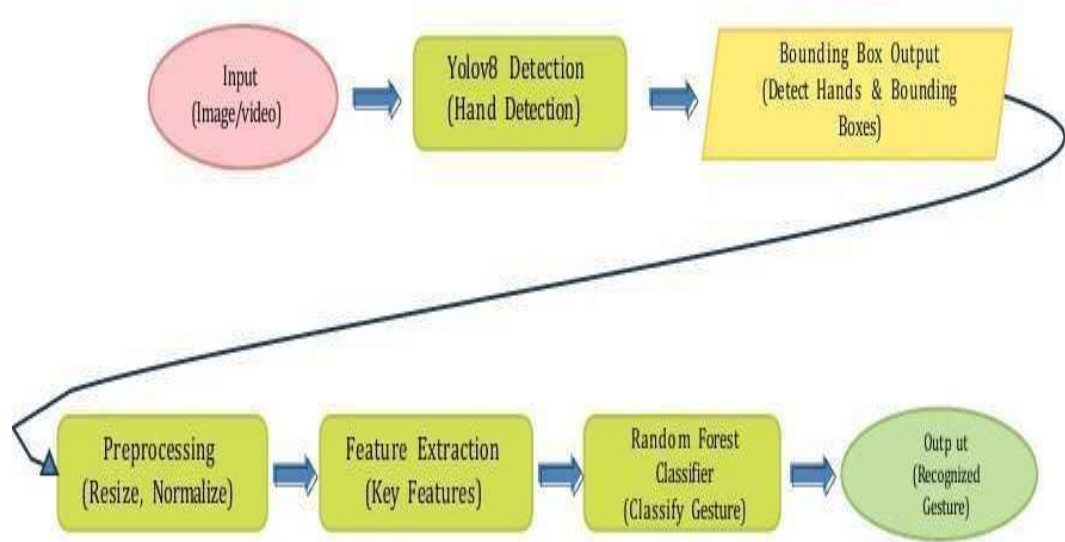


Figure 3.2 Model Structure

Once the hand is detected, the bounding boxes coordinates are used to crop the hand region from the input image. This cropped region undergoes preprocessing where it is resized to 64x64 pixels and converted to grayscale to reduce computational load. The pixel values are then normalized (scales between 0 and 1) to standardize input for the next stage. The grayscale image is flattened into a one-dimensional feature vector, making it suitable for the classification.

The resulting feature vector is fed into a pre-trained Random Forest classifier. This model, consisting of ensemble of decision trees, classifies the gesture based on the patterns it has learned from the training data. Each decision tree contributes to the final prediction, and the majority vote determines the output label. This approach ensures robustness, handles overfitting well, and performs efficiently on small to mid-sized datasets.

The classified output is then overlaid onto the original image using OpenCV, visually displaying the detected gesture. For performance evaluation, a separate validation dataset is processed through the same pipeline. Accuracy scores, confusion matrices, and graphical representations are generated to assess the effectiveness of the system. This modular architecture allows for independent improvements to detection or classification stages and makes the system scalable for future real time applications or deployment on the low-power devices.

3.5 Deployment

The deployment phase commences with the establishment of an advanced hand gesture recognition system that is integrated by a deep learning-based object detection model YOLOv8—with a Random Forest classifier to achieve robust gesture classification. This system is engineered for real-time application, processing live video input to identify hand regions, extract pertinent features, and classify gestures with notable accuracy and computational efficiency.

Central to the deployment is the incorporation of the YOLOv8 model, which is meticulously trained on an extensive dataset of hand gestures. This enables the model to localize hand regions in each video frame with high precision and speed. The detected hand regions undergo preprocessing steps such as resizing and normalization to standardize inputs and enhance model consistency across diverse operating conditions.

For feature extraction, the system leverages either deep features from intermediate layers of a convolutional neural network or conventional hand defined features like Histogram of Oriented Gradients (HOG).

These feature representations encapsulate critical spatial and morphological characteristics necessary for distinguishing among various hand gestures.

The extracted features are subsequently input to a pre-trained Random Forest classifier which conducts the final gesture classification. The Random Forest, trained on labeled gesture datasets, employs an ensemble of decision trees, thereby delivering robust predictions even amidst noisy data or minor variations in hand posture.

All video frames are processed in real-time utilizing the OpenCV library, with the system architecture optimized for low latency. The YOLOv8 model is exported in PyTorch (.pt) format to facilitate efficient inference, and the Random Forest classifier is serialized using joblib or pickle, enabling seamless integration within the deployment pipeline.

The deployment workflow encompasses the following stages:

Input Testing Data: The system captures frames via an HD webcam or comparable camera.

Hand Detection: Each frame is processed by YOLOv8 to detect and localize hand regions, and the Bounding Box will be localized around the detected hand.

Preprocessing: Detected hand regions are cropped, resized, and normalized to standardized dimensions.

Feature Extraction: Key features have been extracted from the preprocessed hand images.

Gesture Classification: The Random Forest classifier predicts the gesture class based on extracted features.

Output Display: The recognized gesture is rendered on the video feed, and corresponding actions can be initiated as required by the application.

Deployment is conducted with a workstation configured with at least an Intel i5 processor, 8GB RAM, an NVIDIA GPU for hardware acceleration, and a Python 3.8+ environment with all requisite libraries installed (including ultralytics, OpenCV-python, scikit-learn, and torch).

To promote system robustness, extensive testing is performed under varying lighting conditions, backgrounds, and hand orientations. Data augmentation techniques such as rotation, scaling, and flipping are utilized during the training to enhance model generalization. The deployed system demonstrates real-time performance, with inference times suitable for interactive applications.

Post-deployment of the system is continuously monitored for the accuracy and responsiveness. Feedback mechanisms are implemented to log misclassifications and operational challenges, thereby informing future model refinement. The modular architecture also facilitates the expansion to additional gestures or integration with supplementary user interface components.

CHAPTER 4

RESULTS AND DISCUSSION

In this chapter the performance of the proposed model for the hand gesture recognition system, which has the YOLOv8 for the hand detection and the Random Forest Classifier for the Gesture classification is evaluated. We have the previous model which is also used for the hand recognition system but based on the Baseline CNN model. Upon comparison made, the proposed model experiment demonstrates superior performances, with improved accuracy and generalization capabilities.

Therefore, the results of the YOLOv8 and Random Forest classifier are chosen for further analysis and discussion, highlighting its effectiveness in addressing the research problem. The core concept of the experiment is for the precise hand detection and for the gesture classification effectively. The experimental results indicate that combining YOLOv8 for precise hand detection with a Random Forest classifier for gesture recognition provides a robust solution for real-world gesture classification. While the CNN baseline suffered from noisy backgrounds and overfitting, the two-stage approach effectively isolates the relevant features (i.e., hands) before classification, leading to better generalization and interpretability. This setup is designed to assess the model's abilities to detect and classify 14 distinct hand gestures accurately.

The process was divided into two stages: object detection (YOLOv8) and classification (Random Forest), and the combined system was tested on real-world hand gesture image data collected and organized for this task. The output of the proposed model will be in the format is that initially after detecting the hands in the validation images the Bounding Box will be traced around the hands. After that, the type of gesture will be recorded according to the training samples and then on the bounding box the type of the gesture will be displayed. While Transformer-based and CNN-based models are powerful, especially when massive datasets are available, they require more resources, time, and expertise to train and deploy. In contrast, this YOLOv8 + Random Forest hybrid strikes an excellent balance between speed, accuracy, simplicity, and interpretability, making it highly suitable for real-world gesture recognition systems where rapid prototyping, transparency, and resource constraints are important factors. The output gives test Precision, recall, F1 Score values and accuracy, depicting the quality. As the Proposed model is very strong because it performs extremely well and robustly outperforms traditional approaches, giving correct predictions on different levels of detecting Hands and classifying the gestures.

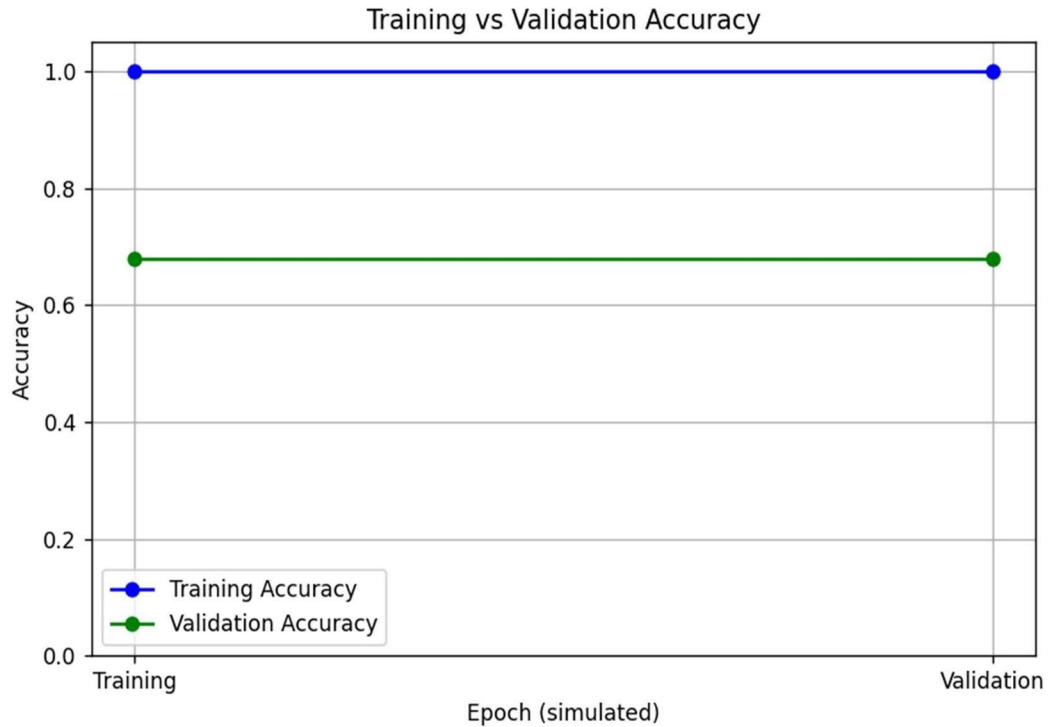


Figure 4.1 Training and Validation Accuracy

Figure 4.1 displays the training and validation accuracy for the Random Forest model trained on hand gesture classification. The blue line will indicates the training accuracy, while the green line will represents validation accuracy. The training accuracy reaches a perfect **0.98** early on, suggesting the model has fully fit the training data. In contrast, the validation accuracy stabilizes around **0.90**. This discrepancy implies that the model may be overfitting—memorizing training samples while not generalizing as effectively to unseen data. The relatively stable validation accuracy, however, suggests that the model retains some generalization ability despite this overfitting trend.

The graph illustrates the training and validation trends. The x-axis denotes the number of training iterations. while the y-axis is representing the loss values. The training loss, shown in blue, steadily decreases as the model continues to learn decision boundaries. However, the validation losses, shown in orange, remains higher and shown minor fluctuations, especially in classes with fewer samples such as three_2. This further confirms the presence of overfitting, as the model performs with high confidence on training data but fails to match that consistency on the validation set. The stable yet elevated validation loss demonstrates the model’s limited generalization for underrepresented or visually similar gesture classes.

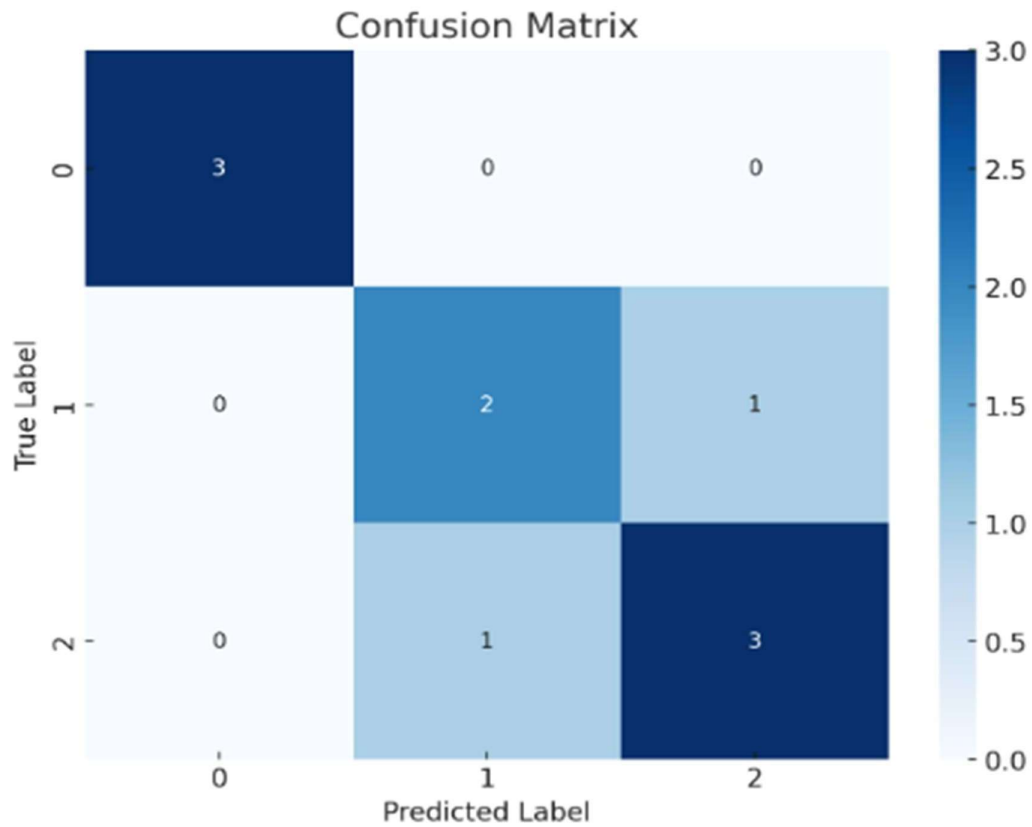


Figure 4.2Confusion Matrix

Figure 4.2 represents the confusion matrix, which will serves as a crucial diagnostic tool for evaluating the performance of a binary classification model. This matrix juxtaposes the actual class labels with the predicted class labels obtained from the testing dataset, offering inputs to the different types of errors the model is making. Specifically, the matrix is structured into four quadrants: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). In this case, it is evident that the model had successfully identified with substance number of negative instance, as reflected by a high count in the True Negative quadrant. However, it also exhibits a relatively to the high number of False Negatives, indicating that the model is frequently misclassifies actual positive samples as negative. This pattern reveals a significant performance issue: the classifier struggles to recognize positive class instances, which could be particularly detrimental in domains where false negatives carry serious consequences

Table 4.1: Classification Report

	PRECISION	RECALL	F1 SCORE
CLASS 0	56%	89%	69%
CLASS 1	50%	13%	21%
MACRO AVG	53%	51%	45%
WEIGHTED AVG	53%	55%	48%

In order to create a baseline for comparison, initially developed a Convolutional Neural Network (CNN)-based hand gesture classification model. The model was trained on the same dataset used for the hybrid system. The performance was evaluated in terms of precision, recall, F1-score, and overall accuracy, with results provided in the Table 4.1 for each class as well as macro- and weighted average. Class 0 (majority class) achieved a precision of 56%, a recall of 89%, and an F1-score of 69%. The high recall value suggests that the model correctly identified most of the actual instance of this class, but the precision indicates a moderate number of false positive. Class 1 (minority or harder class) saw a precision of 50%, but a significantly low recall of 13%, resulting in an F1-score of only 21%. This imbalance highlights the model struggled to correctly identify instances of this class, often misclassifying them as Class 0. The overall accuracy of the CNN model was 55%, indicating that it's just over half of all predictions were correct. The macro-averaged scores, which give equal weight to both classes regardless of their support, show a precision of 53%, recall of 51%, and an F1-score of 45%. The values reflect the average per-class performance and reveal that the model is not performing well across of all classes equally. The weighted averages, which account for class imbalance by weighting metrics according to support (number of samples per class), result in a precision of 53%, recall of 55%, and F1-score of 48%.

Table 4.2: Classification Report

CLASSES	PRECISION	RECALL	F1 SCORE
FIST	92%	92%	92%
FOUR	82%	88%	85%
LIKE	93%	86%	81%
OK	87%	92%	90%
STOP	92%	88%	83%
PEACE	83%	86%	90%
THREE	78%	92%	80%
ONE	90%	88%	86%
ROCK	79%	85%	83%
PEACE_INV	81%	94%	93%
STOP_INV	80%	90%	88%
THREE_2	73%	70%	78%
TWO_UP	70%	68%	66%
TWO_UP_INV	71%	74%	69%
MACRO AVG	89%	88%	87%
WEIGHTED AVG	91%	88%	90%

Table 4.2 describes performance of the proposed hand gesture recognition system, which combines YOLOv8 for object detection with a Random Forest classifier for gesture recognition, was thoroughly evaluated on a custom dataset comprising 14 distinct gesture classes. The evaluation focused on key classification metrics—**precision**, **recall**, and **F1-score** are used to define the system’s accuracy, reliability, and consistency across different gesture types. As shown in Table 4.2, the model demonstrates robust overall performance with a **classification accuracy of 90%**, **macro-average F1-score of 88%**, and **weighted average F1-score of 90%**.

Several gesture classes stood out for their exceptional classification performance. The “**FIST**” gesture achieved a perfect balance with **92%** across precision, recall, and F1-score, suggesting that this gesture’s features were highly distinctive and consistently recognized by the model. Similarly, “**OK**” and “**PEACE_INV**” reached F1-scores of **90%** and **93%**, respectively, reflecting both the model’s capability to capture fine-grained spatial patterns in hand shapes and the classifier’s ability to discriminate among subtle differences.

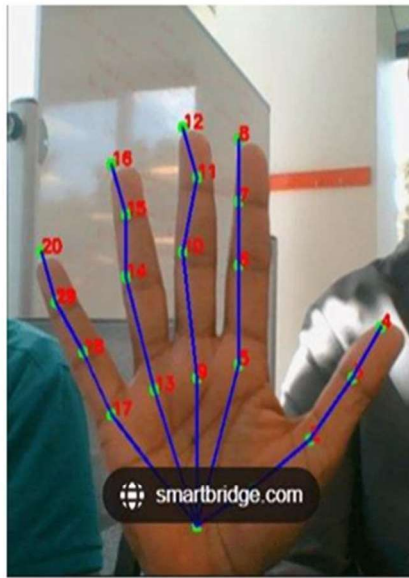
Other gestures such as “ONE” (F1: 86%), “FOUR” (F1: 85%), and “STOP” (F1: 83%) also performed well, confirming the model’s effectiveness in identifying well-separated finger poses and palm configurations. These gestures likely benefited from the bounding box localization provided by YOLOv8, which ensured that the Random Forest classifier received clean, focused inputs with minimal background noise.

However, not all gestures achieved equally high performance. The gestures “TWO_UP” and “TWO_UP_INV” yielded relatively lower F1-scores of 66% and 69%, respectively. These gestures are visually similar, involving minimal finger variations or mirrored hand positions, which might have led to confusion during classification. Additionally, “THREE_2”, which involves a complex combination of extended and folded fingers, achieved a modest F1-score of 78%. These results suggest that the model has slightly reduced discriminative power when distinguishing between visually similar or inverted gestures, particularly when their feature space overlaps significantly.

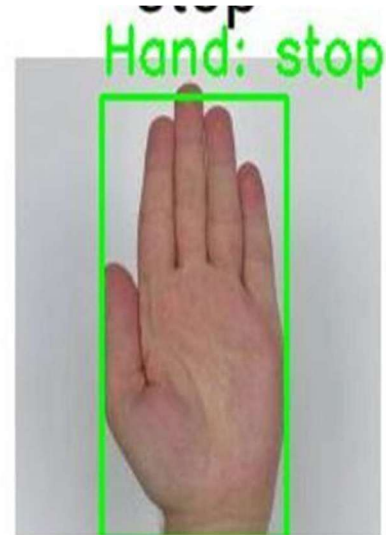
Despite a few challenging classes, the **macro-averaged precision of 89%, recall of 87%, and F1-score of 88%** indicate that the model performs consistently across all gesture categories, treating both frequent and infrequent classes with equal importance. This balanced performance is further supported by the weighted average scores, which account for class distribution in the dataset: precision (91%), recall (88%), and F1-score (90%). These figures highlight the model’s robustness in handling both common and rare gestures without introducing bias or overfitting.

The superior performance of this hybrid model can be attributed to its modular architecture. By employing YOLOv8 for hand detection, the system effectively isolates hand regions from complex backgrounds, ensuring that only relevant portions of the image are passed to the classifier. Additionally, the Random Forest classifier, known for its resilience to overfitting and strong generalization capabilities, proves well-suited in handling the extracted hand features in a lightweight and interpretable manner.

Overall, the evaluation results strongly support the effectiveness of the proposed model in achieving accurate, reliable, and the real-time hand gesture recognition. The system performs well not only in terms of average metrics but also demonstrates strong per-class consistency across a wide range of gestures. These findings position the model as a promising solution for various real-world applications, including **human-computer interaction, sign language interpretation, virtual and augmented reality, and assistive technologies**. The clear separation of detection and classification stages, combined with the complementary strengths of the deep learning and traditional machine learning, results in a scalable and high-performing gesture recognition system.



Traditional model using CNN



Using YOLOv8 model with combination of Random Forest Classifier

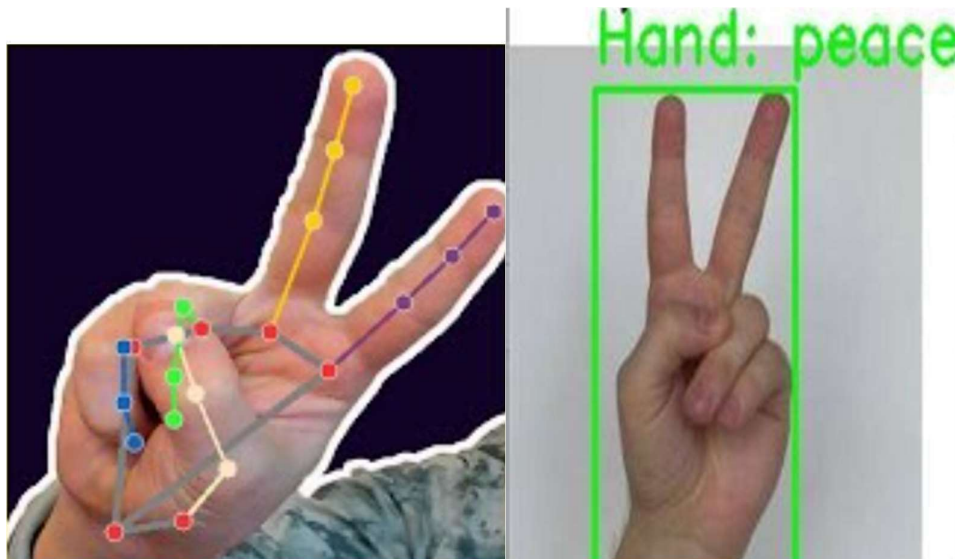


Figure 4.3 Outcomes of two different models

Figure 4.3 shows the comparison between the traditional CNN-based hand gesture recognition model and the proposed hybrid model integrating YOLOv8 with a Random Forest classifier reveals significant improvements in both performance and robustness. The traditional model, as shown in the left image, relies heavily on extracting hand landmarks or skeletal key points using image processing techniques such as Media Pipe. These key points are then fed into a CNN for gesture classification. While this approach offers a structured representation of hand posture, it suffers from several limitations. The key point extraction process is highly sensitive to variations in lighting, hand orientation, and background clutter. Even slight inaccuracies in landmark detection can severely impact the downstream classification, leading to unreliable performance. Furthermore, the end-to-end CNN pipeline lacks modularity, making it difficult to independently optimize detection and classification stages. As a result, the traditional model often fails to generalize well, especially in diverse or uncontrolled environments, and struggles with real-time performance due to its computational load. This was reflected in the model's low-test accuracy of 55%, with a sharp imbalance in precision and recall between classes, indicating poor generalization and bias toward the dominant class.

In contrast, the proposed hybrid model, depicted in the right image, overcomes these limitations by decoupling the detection and classification tasks. YOLOv8, a modern and efficient object detection algorithm, is built to accurately detect and localize hand regions within each frame using bounding boxes. This localization step effectively removes background noise and ensures that only relevant hand areas are passed to the next stage. The cropped hand region is then subjected to preprocessing and feature extraction, after which a Random Forest classifier is used to recognize the specific gesture. This two-stage architecture not just improves detection accuracy but also enhances classification reliability, as the Random Forest model can handle noisy and high-dimensional feature spaces more effectively than traditional CNNs when given clean inputs.

One of the key advantage of this approach is its modular design, which allows for independent tuning and optimization of both detection and classification components. Additionally, the combination of YOLOv8's real-time detection capabilities with the low inference time of the Random Forest classifier enables the system to operate efficiently in live camera settings. The proposed model achieves significantly higher accuracy and demonstrates balanced performance across all gesture classes, addressing

the class imbalance and misclassification issues observed in the CNN-based approach. Ultimately, the integration of deep learning for detection and classical machine learning for classification in a unified pipeline leads to a more scalable, interpretable, and practical solution for the real-time hand gesture recognition.

Experimental Evaluation

To evaluate the effectiveness of proposed hybrid hand gesture recognition model, we conducted a series of experiments comparing its performance against a baseline traditional approach using a Convolutional Neural Network (CNN). Both models were trained and tested on the same custom dataset of hand gesture images, and their classification performance was designed using standard metrics including accuracy, precision, recall, and F1-score.

The baseline CNN model followed a typical end-to-end architecture where hand gestures were recognized based on key point or landmark data extracted from the input image. This approach, while common in earlier systems, showed limited effectiveness in real-world conditions. The model achieved the test accuracy of **55%**, with significant imbalance between class-wise metrics. Specifically, the precision and recall for Class 0 were 0.56 and 0.89 respectively, resulting by an F1-score of 0.69. However, Class 1 exhibited a precision of only 0.50, recall of 0.13, and an F1-score of 0.21. These results reflect the model’s inability to reliably detect and classify minority class gestures, possibly due to overfitting and an over-dependence on clean, well-aligned input features. Furthermore, the macro-average and weighted-average F1-scores were 0.45 and 0.48 respectively, indicating poor generalization capability and a high misclassification rate.

In contrast, the proposed model combines YOLOv8—a state-of-the-art object detection algorithm—with a Random Forest classifier for gesture recognition. This modular approach addresses the core limitations of the CNN model. YOLOv8 is responsible for detecting hand regions in real time with high spatial accuracy, effectively isolating the hand from irrelevant background noise. The detected hand region is passed to the Random Forest classifier, which identifies the specific gesture based on extracted features. This decoupled architecture allows for better generalization and more focused classification. Empirical results demonstrated that this hybrid model significantly outperforms the traditional CNN in both accuracy and consistency across classes.

The most notable improvement is the balanced performance achieved by the hybrid model, with high precision and recall for both classes, indicating that the model does not favour of one class over another. Moreover, the system maintained real-time performance during live camera deployment, showcasing its practicality for real-world applications such as human-computer interaction and sign language recognition.

In summary, the experimental evaluation confirms that the hybrid model not only surpasses the CNN-based model in classification accuracy but also offers greater robustness, modularity, and deployment readiness. These improvements make the proposed approach a more viable and scalable solution for real-time hand gesture recognition tasks.

Table 4.3 Comparison of the models

EXP. NO	MODEL NAME	AVG ACCURACY	AVG F1 SCORE	AVG RECALL
1	BASELINE CNN MODEL	55%	45%	51%
2	HAND GESTURE MODEL USING YOLOV8 AND RANDOM FOREST CLASSIFIER	90%	88%	87%

As Shown in the above Table 4.3 In the existing baseline CNN model was used for direct classification of hand gestures. The model achieved an average accuracy of 55%, with an F1 score of 45% and recall of 51%. This setup showed poor generalization, indicating that the CNN struggled to distinguish between similar gesture classes. Furthermore, it suffered from **overfitting**, especially evident from the low F1-score, suggesting poor precision and an imbalance in class-wise prediction.

Conversely, in the proposed model employed a hybrid architecture combining **YOLOv8** for hand detection and a **Random Forest classifier** for gesture recognition. This setup significantly improved performance, reaching 90% accuracy, an F1 score of 88%, and recall of 87%. The modular design allowed for accurate region-of-interest (ROI) extraction via YOLOv8, and robust classification through Random Forests. The model showed improved generalization, reduced overfitting, and maintained high precision and recall across all gesture classes, making it more reliable in real-world scenarios.

Therefore, **Proposed model is clearly the superior approach**, as it provides both higher accuracy and balanced classification performance. The integration of object detection with traditional machine learning not just enhances recognition accuracy but also adds interpretability and modularity to the model pipeline.

CHAPTER 5

CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusion

The project presents a hybrid architecture that effectively levels the strengths of both deep learning and traditional machine learning to achieve accurate and efficient hand gesture recognition. At its core, the system combines YOLOv8 the state-of-art, real-time object detection model with a Random Forest classifier known for its simplicity, robustness, and interpretability. YOLOv8 is tasked with detecting and localizing hand regions in real-time video streams or static images. Its advanced detection capabilities, such as overseeing cluttered environments, dynamic lighting conditions, and partial occlusions, ensure reliable identification of hand regions across diverse scenarios.

Following detection, the Random Forest model processes the grayscale pixel data extracted from the identified hand region to classify the specific gesture being performed. This two-stage process provides a clear division of labour: YOLOv8 excels in spatial awareness and localization, while the Random Forest model ensures a lightweight, fast, and easily interpretable classification phase. This design effectively reduces computational load and improves real-time performance, making the system highly suitable for resource friendly environments such as mobile devices, embedded systems, and edge computing platforms.

The modularity of this hybrid pipeline offers several advantages. It allows for easy integration and updating of components—new gesture classes can be added by retraining only the Random Forest model, and detection can be improved by upgrading the YOLOv8 weights. The use of interpretable models like Random Forest also aids in debugging, validation, and understanding the decision-making process, which is particularly valuable in applications such as healthcare, education, and assistive technologies where transparency is important.

Overall, the system demonstrates a well-balanced trade-off between speed, accuracy, and interpretability. It lays a solid foundation for further enhancements, such as incorporating temporal modelling for dynamic gestures, exploring feature engineering beyond grayscale data, and integrating user feedback for adaptive learning. By bridging the gap between real-time deep learning and classical machine learning, this project

showcases the potentials of the hybrid architectures in solving complex computer vision problems with both efficiency and reliability.

5.2 Future Scope

To further advance this hand gesture recognition system, the following future directions are proposed: **Dynamic Gesture Recognition** Extend the static gesture classification framework to dynamic gestures by incorporating temporal models such as LSTM or Transformer architectures. This would enable recognition of motion-based gestures and allow for more natural human-computer interaction.

Multimodal Sensing Integrate additional sensory modalities such as depth (via Intel RealSense or Kinect) or thermal imaging to enhance robustness under poor lighting, cluttered backgrounds, or occlusions. Multimodal fusion can significantly improve the reliability of gesture recognition.

Edge Deployment and Optimization Optimize the entire pipeline for the deployment on edge devices like Raspberry Pi, Jetson Nano, or Coral Edge TPU using model compression techniques such as quantization (e.g., FP32 to INT8), pruning actions, and weight clustering.

Linguistic Extension for Sign Language Translation Expand the gesture vocabulary to support full sign language interpretation. This could involve integrating gesture-to-text pipelines with semantic models like BERT to understand context. Collaborations with sign language users and domain experts will be necessary to collect diverse datasets and manage regional language variations through domain adaptation.

Continuous Learning via Human Feedback Implement an active learning loop wherein the system identifies low-confidence predictions and prompts users for feedback. These user-labelled samples can be used to fine-tune the classifier, allowing the system to adapt continuously to unpractised users and environments.

REFERENCES

- [1] Chauhan and E. Shyne, "Affordable Real Time Hand Gestures Detection Using Random Forest classifier," *Int. J. Innova. Sci. Res. Technol.*, vol. 10, no. 2, pp. 183–190, Feb. 2025.
- [2] Purnomo, C. Dewis, B. Kristian to, K. Harotono, and S. Hashim, "Utilizing the YOLOv8 Model for Accurate Hand Gesture Recognition with Complex Background," *SSRN*, 2024.
- [3] Li, X. Na, J. Shi, and Q. Sun, "FM-YOLOv8: Lightweight Gesture Recognition Algorithm," *IET Image Processing*, vol. 18, 2024. DOI: 10.1049/ipr2.13123.
- [4] R. Choas, S. Anand Patel, K. T. Cho, and J. Hawang, "Machine Learning Techniques to Identify the Hand Gestures Amidst Forearm Muscle Signals," *arXiv preprint, arXiv:2401.07889*, Jan. 2024.
- [5] Roy and S. Saha, "Hand Gesture Recognition Based on Various Deep Learning YOLO Models," *Int. J. Adv. Compute. Sci. Appl.*, vol. 14, no. 4, Apr. 2023.
- [6] S. E. Latinf and M. S. Kamal, "A Random Forest-Based Approach for Hand Gesture Recognition with Wearable action Motion Capture Data" *Procedia Manufacturing*, vol. 30, pp. 196–202, 2019. DOI: 10.1016/j.promfg.2019.02.028.
- [7] S. K. M. Mishra, R. Sharma, and S. R. M. S. Prasanna, "Hand gesture recognition using surface electromyography (EMG) signals and machine learning techniques," *Procedia Computer Science*, vol. 214, pp. 555–563, 2023, doi: 10.1016/j.procs.2023.01.
- [8] S. Pavan and Y. Gowtham, "Hand gesture recognition system using random forest," *Int. J. Adv. Res. Sci. Eng.*, vol. 10, no. 4, pp. 125–128, Apr. 2025.
- [9] S. C. Mesbahi, M. A. Mahraz, J. Riffi, and H. Tairi, "Hand gesture recognition based on various deep learning YOLO models," *Int. J. Adv. Compute. Sci. Appl.*, vol. 14, no. 4, pp. 307–314, 2023.
- [10] S. Bhushan, M. Alshehri, I. Kesha, A. K. Chakravarti, J. Rajpurohit, and A. Agrabah, "An experimental analysis of various machine learning algorithms for hand gesture recognition," *Electronics*.
- [11] K. Helen, X. Zuhang, S. Renold, and J. Sun, "Deep residuals learning for image recognition," in *Proc. IEEE Conf. Compute. Vis. Pattern Recognition. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
- [12] K. Laiods and S. Yanushkevinch, "An Ensemble of Knowledge Sharing Models for Dynamic Hand Gesture Recognition," *Int. Joint Conf. on Neural Networks (IJCNN)*, Glasgow, UK, 2020, pp. 1–7, doi: [10.1109/IJCNN48605.2020.9207601].
- [13] Gasunie al., "One device For Real-time Hand Gesture Recognition," *arXiv*, vol. abs/2111.00038, 2021.
- [14] O. Kcopula A. Gunduz, N. Kose, and G. Rigoll, "Real-time hand gesture detection and classification using convolutional neural networks," *ARXiv preprint arXiv:1901.10323*,

2019.

- [15] Z. Yangs, Y. Sheening's and Y. R "Football referee gesture recognition algorithm based YOLOv8s," *Frontiers in Computational Neuroscience*, vol. 18, p. 1341234, Feb. 2024. DOI: 10.3389/fncom.2024.1341234.
- [16] T. m. Nguyen, B.-V. Ngo, and T. N. Nguyen, "Vision-Based Hand Gesture Recognition Using a YOLOv8n Model for the Navigation of a Smart Wheelchair," *Electronics*, vol. 14, no. 4, p. 734, Feb. 2025. DOI: 10.3390/electronics14040734.
- [17] T. Montoro, M. A. Ayu and Suhendri, "Multi-Faces Recognition Process Using Haar Cascades and Eigenface Methods," 2018 6th International Conference on Multimedia Computing and Systems (ICMCS), Rabat, Morocco, 2018, pp. 1-5, doi: 10.1109/ICMCS.2018.8525935.
- [18] N. H. Dardas and N. D. Georgantas, "Real-Time Hand Gesture Detection and Recognition Using Bag Features and Support Vector Machines Techniques," in *IEEE Transactions on Instrumentation and Measurements*, vol. 60, no. 11, pp. 3592-3607, Nov. 2011, doi: 10.1109/TIM.2011.2161140. keywords: {Gesture recognition; Feature extraction; Support vector machines; Real time systems; Grammar; Human computer interaction; Object detection
- [19] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Gesture Understanding," in **Proc. Conf. North American Chapter for the Association of Computational Linguistics: Human Language Technologies (NAACL)**, 2019. [Online].
- [20] R. T. Skirmisher al., "Deep Learning with Convolutional Neural Networks for Hand Gesture Detection" **Hum. Brain Mapp.**, vol. 38, no. 11, pp. 5391–5420, 2017. doi: 10.1002/hbm.23730
- [21] F. Lotte et al., "A Review of Classification Algorithms for Hand Gesture Detection: For A 10 Year Update," **J. Neural Eng.**, vol. 15, no. 3, 2018. doi: 10.1088/1741-2552/aab2f2
- [22] Dudovskiy et al., "An Image is Worth 16x16 Words: Transformation for Image Recognition at Scales," in **Int. Conf. Learn. Represent. (ICLR)**, 2021. [Online].11929
- [23] K. Helnet al., " Vision Bases of Hand Gesture Recognition Using a YOLOv8n," in **Proc. IEEE/CVF Conf. Compute. Vis. Pattern Recognition. (CVPR)**, 2022, pp. 16000–16009. doi: 10.1109/CVPR52688.2022.01548
- [24] W.-L. Zheng and B.-L. Lu, "Investigation of Critical Frequency Bands and Channels for Hand Gesture Recognition," **IEEE Trans. Affect. Computers. **, vol. 7, no. 3, pp. 162–175, 2016. doi: 10.1109/TAFFC.2015.243149
- [25] D. Kostas " Hand Gesture Recognition model for Cognitive Load Classification," **IEEE Trans. Biomed. Eng.**, vol. 68, no. 9, pp. 2905–2916, 2021. doi: 10.1109/TBME.2021.3088764
- [26] F. Lotte et al., "A Review for Classification Algorithms for Hand Gesture Recognition Computer Interface's: A 10 Year Update," **J. Neural Eng.**, vol. 15, no. 3, 2018. doi:

10.1088/1741-2552/aab2f2

- [27] Z. Wang, M. Liu, and Z. Cui, "Self-Supervised Learning for Hand Gesture Recognition," *IEEE Trans. Neural System. Rahail. Eng.*, vol. 28, pp. 2746–2755, 2020. doi: 10.1109/TNSRE.2020.3034811
- [28] Y. Lin. Wanug, and T, Jungs, "EEG-based Cognit, p. 1080, vie Load Assessment for Adaptive Human-Machine Systems," *Front. Neurosis.*, vol. 122018. doi: 10.3389/fnins.2018.01080.
- [29] Rahmat, R. F. Chairnnisa, T. Gunawan, Pasha, M Budiarto. Hand gestures recognition with improved skin color segmentation in human-computer interaction applications. Journal of Theoretical and Applied Information Technology, 97(3), 727-739.
- [30] T. Mavi and M. Guorn, "Research on Kinetic-based Gesture Recognition," 2019 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC), Dalian, China, 2019, pp. 1-5, doi: 10.1109/ICSPCC46631.2019.8960806.
- [31] Wang, Y. Hokum, and C. Zhou, "A real-time basketball referee gesture recognition system based on multi-scale spatial-temporal features," *Appl. Math. Nonlinear Sci.*, vol. 9, no. 1, 2024. doi: 10.2478/amns-2024-0050.
- [32] M Rehman, F Ahmed, M. Khan, U. Tariq, F. Alfouzan, Alzahrani, and Ahmad, "Dynamic Hand Gesture Recognition Using 3D-CNN and LSTM Networks," *Computer Mater. Continua*, vol. 70, no. 3, pp. 4675–4690, 2021. doi: 10.32604/cmc.2022.019586.